Міністерство освіти і науки України

Національний технічний університет України

“Київський політехнічний інститут імені Ігоря Сікорського”

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Лабораторна робота №2

Методи і технології паралельного програмування

Тема: Засоби створення та керування потоками в паралельних мультипоточних програмах

Виконав:

студент групи ІП-51мн

Панченко С. В.

Перевірив:

Лесик В. О.

Київ 2026

ЗМІСТ

1 Мета роботи . . . 3
2 Завдання до виконання . . . 4
3 Обґрунтування обрання фреймворку . . . 5
4 Короткий опис рішень, критичні моменти та гіперпараметри . . . 6
4.1 Порівняння Map-Reduce, Fork-Join, Worker Pool . . . 6
4.1.1 Аналіз HTML-документів . . . 6
4.1.2 Обробка масиву чисел . . . 6
4.1.3 Множення матриць . . . 7
4.2 Pipeline та Producer-Consumer . . . 8
5 Контрольні запитання . . . 10
5.1 Поясніть різницю між м'ютексом та семафором. Коли використовувати кожен з них? . . . 10
5.2 Що таке потокобезпечні (thread-safe) структури даних? Наведіть приклади. . . . 10
5.3 Поясніть різницю між песимістичною та оптимістичною синхронізацією. . . . 10
5.4 Що таке lock-free та wait-free алгоритми? . . . 10
5.5 Поясніть патерни Producer-Consumer, Pipeline, Map-Reduce, Fork-Join, їх реалізацію. . . . 10
5.6 Поясніть можливі відмінності в реалізації патернів відносно різних фреймворків. . . . 11
6 Висновки . . . 12
А Лістинг коду . . . 13
А.1 src/bin/html.rs . . . 13
А.2 src/bin/matrix.rs . . . 15
А.3 src/bin/numbers.rs . . . 19
А.4 src/bin/video.rs . . . 23
А.5 src/bin/wikipedia.rs . . . 26

1 МЕТА РОБОТИ

Ознайомитися з механізмами взаємодії та синхронізації потоків, набути практичних навичок реалізації паралельних алгоритмів з координацією роботи між потоками за допомогою використання відповідних патернів паралельного програмування.

2 ЗАВДАННЯ ДО ВИКОНАННЯ

1. Основне завдання: Реалізувати послідовну та паралельну мультипоточну обробку незалежних задач. Обрати відповідний фреймворк, описати його можливості, обмеження та причину вибору.
2. Задача 1: Порівняння патернів Map-Reduce, Fork-Join, Worker Pool на задачах:
    - Підрахунок частоти появи тегів у наборі HTML документів (> 1000 файлів).
    - Пошук мінімуму, максимуму, медіани та середнього значення масиву із > 1 000 000 чисел.
    - Множення двох великих матриць (> 1000 чисел).
3. Задача 2: Реалізувати Pipeline та Producer-Consumer для задачі обробки зображень (декодування, фільтр Grayscale, водяний знак, кодування).
4. Дослідження: Проаналізувати швидкодію за різних гіперпараметрів, порівняти підходи, зробити висновки.

3 ОБҐРУНТУВАННЯ ОБРАННЯ ФРЕЙМВОРКУ

Для виконання лабораторної роботи було обрано мову програмування Rust та бібліотеку для data-паралелізму Rayon. Додатково використовувалась стандартна бібліотека `std::thread` та канали `std::sync::mpsc`.

Можливості та переваги:

- Безпека пам'яті: Строга система типів та модель володіння (Ownership) Rust гарантують відсутність гонок даних (data races) на етапі компіляції.
- Rayon (Work-Stealing): Використовує надзвичайно ефективний планувальник, де кожен потік має власну чергу завдань, але може "красти" роботу в інших потоків у разі простою. Ідеально підходить для патернів Map-Reduce (через паралельні ітератори) та Fork-Join (через `rayon::join`).
- Scoped Threads: Використання `std::thread::scope` дозволило безпечно запозичувати локальні змінні у робочі потоки (Worker Pool) без оверхеду на їх клонування або використання `Arc`.

Обмеження:

- Rayon оптимізовано під суто обчислювальні (CPU-bound) задачі. Для задач із дуже інтенсивним асинхронним мережевим вводом-виводом він підходить гірше, ніж спеціалізовані асинхронні середовища (наприклад, Tokio).
- Канали стандартної бібліотеки (`mpsc`) дозволяють передавати дані між етапами (Pipeline), але вимагають ручного управління їх закриттям, інакше виникає ризик взаємного блокування (Deadlock).

4 КОРОТКИЙ ОПИС РІШЕНЬ, КРИТИЧНІ МОМЕНТИ ТА ГІПЕРПАРАМЕТРИ

4.1 Порівняння Map-Reduce, Fork-Join, Worker Pool

4.1.1 Аналіз HTML-документів

- Підхід: Парсинг документів за допомогою регулярних виразів.
- Критичний момент: Злиття великої кількості хеш-таблиць. У Worker Pool агрегація дуже ефективно відбувалася “на льоту” через канал до головного потоку.
- Результати: Усі паралельні підходи прискорили виконання приблизно в 6.5 разів (0.16с проти 1.04с послідовно). Map-Reduce виявився найбільш лаконічним у реалізації.

На рисунку 4.1 зображено результати.

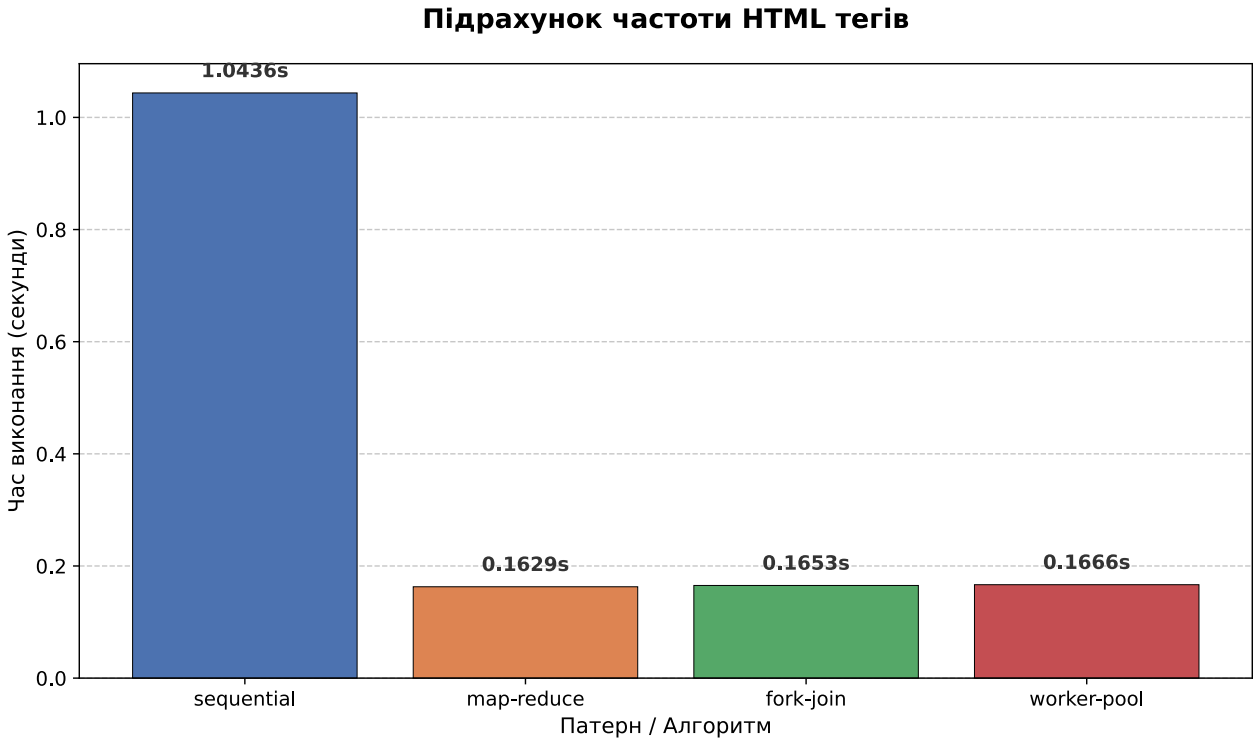

Рисунок 4.1 — Підрахунок частоти html-тегів

4.1.2 Обробка масиву чисел

- Підхід: Пошук статистики виконується за один прохід по чанках масиву.
- Критичний момент: Пошук медіани вимагає сортування. Використання послідовного сортування в Worker Pool зводило загальний час до рівня послідовного алгоритму (3.45с). Заміна на паралельне сортування `par_sort_unstable_by` зменшила час до 0.95с, що яскраво ілюструє закон Амдала.

- Результати: Fork-Join та Map-Reduce відпрацювали найшвидше з ідентичним часом (0.95с).

На рисунку 4.2 зображено результати.

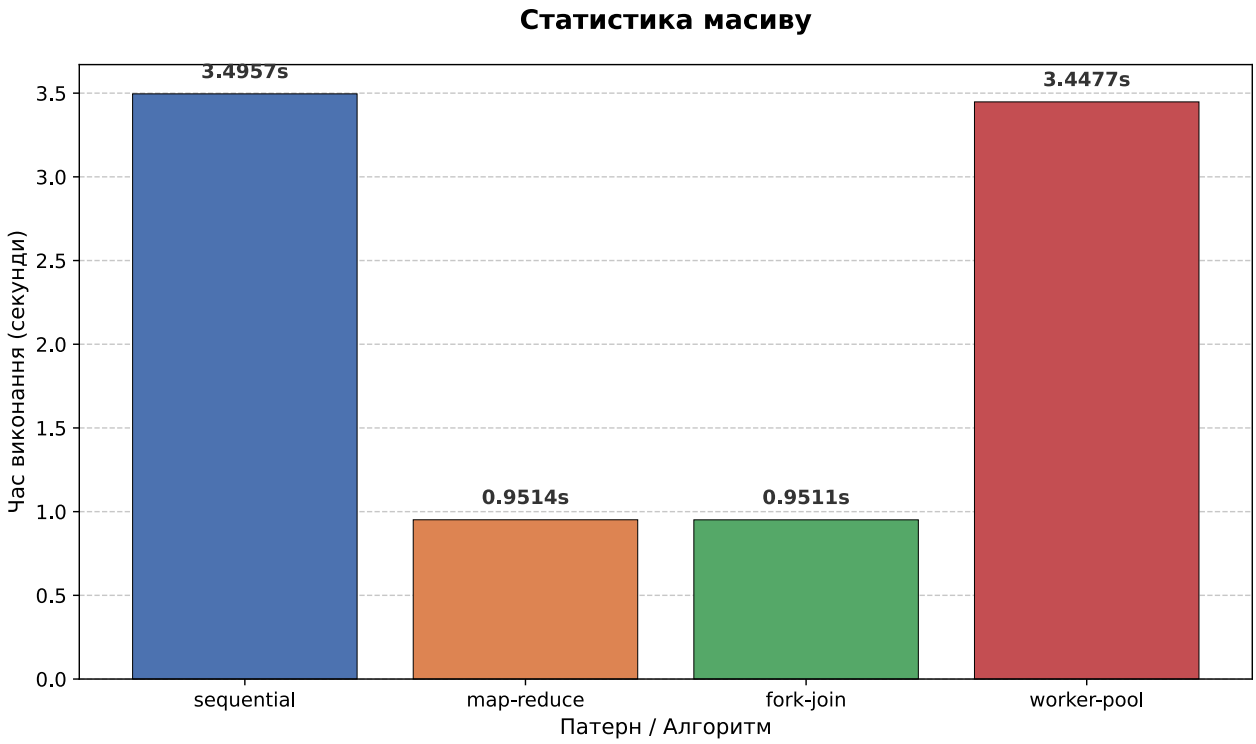

Рисунок 4.2 — Обробка масиву чисел

4.1.3 Множення матриць

- Підхід: Розпаралелювання по рядках результуючої матриці.
- Гіперпараметри: Для Fork-Join підібрано поріг рекурсії (16 рядків), щоб накладні витрати на керування потоками не перевищили користь від паралелізму. Для Worker Pool кількість потоків дорівнювала кількості логічних ядер ОС.
- Результати: Worker Pool показав найкращий час. Обчислення рядків займають однаковий час, що дозволило пулу потоків працювати рівномірно і без простоїв.

На рисунку 4.3 зображено результати.

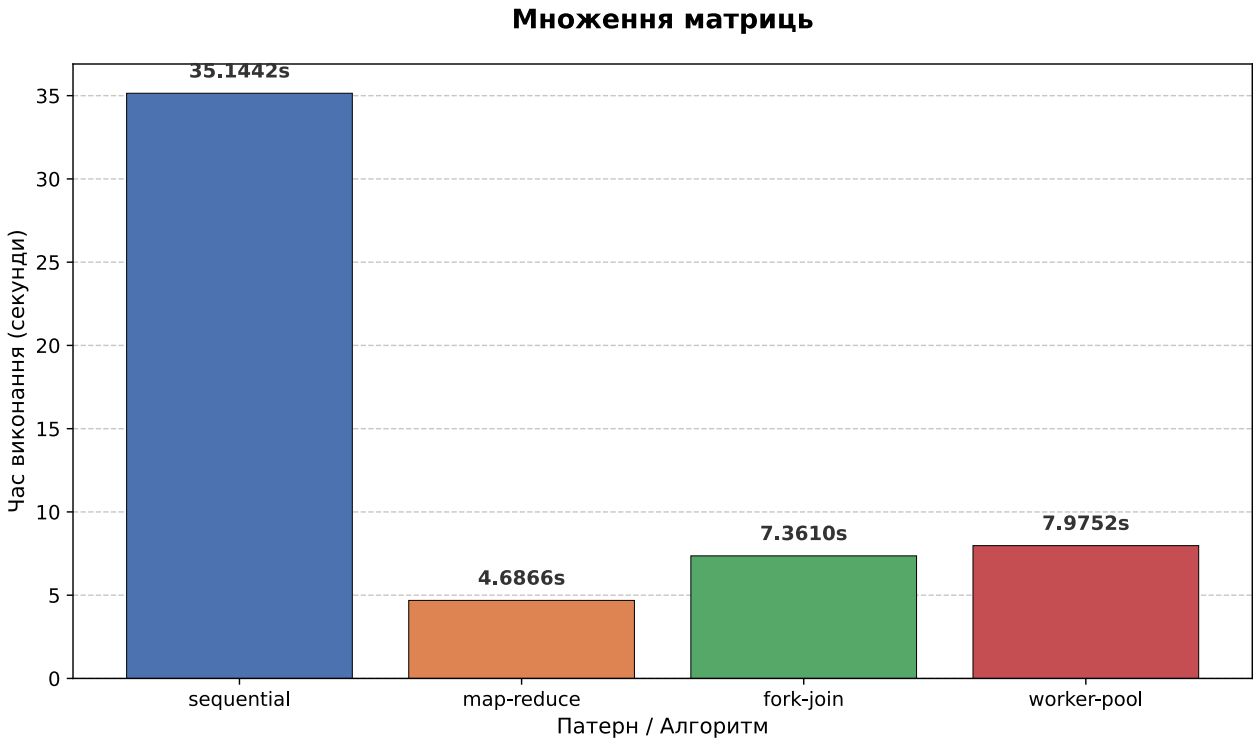

Рисунок 4.3 — Множення матриць

4.2 Pipeline та Producer-Consumer

- Підхід: Оброблялося 100 кадрів. Producer-Consumer використовував одну чергу mpsc, з якої потоки брали файл і виконували весь цикл операцій. Pipeline складався з 3 стадій (Читання -> Фільтр -> Запис), з'єднаних каналами `sync_channel`.
- Критичні моменти: У Pipeline виникало взаємне блокування (Deadlock) через незакриті канали. Явне видалення передавача `std::mem::drop(tx1)` вирішило проблему каскадним закриттям стадій. Також використання обмежених каналів (буфер на 10 елементів) створило зворотний тиск і запобігло переповненню оперативної пам'яті нестиснутими зображеннями.
- Результати: Producer-Consumer відпрацював швидше через відсутність витрат на пересилку масивів пікселів між стадіями. Проте Pipeline є архітектурно доцільнішим, якщо різні стадії вимагають різних ресурсів (наприклад, дисковий ввід-вивід проти обчислень на GPU).

На рисунку 4.4 зображено результати.

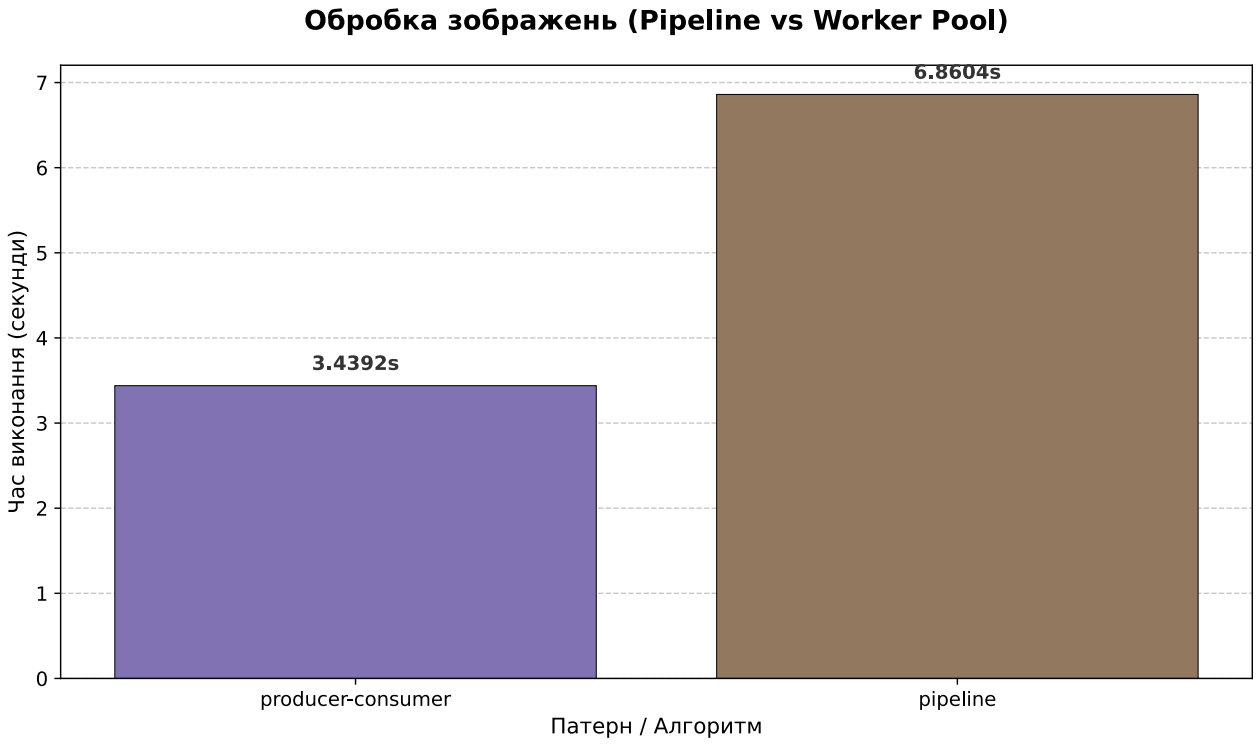

Рисунок 4.4 — Обробка зображень

5 КОНТРОЛЬНІ ЗАПИТАННЯ

5.1 Поясніть різницю між м'ютексом та семафором. Коли використовувати кожен з них?

М'ютекс (Mutex) гарантує ексклюзивний доступ до ресурсу лише для одного потоку в певний момент часу. Він має поняття "власника". Семафор — це лічильник, який дозволяє доступ до ресурсу заданій кількості потоків одночасно. М'ютекс використовується для захисту спільних змінних (наприклад, лічильників), а семафор — для управління пулом ресурсів (наприклад, обмеження кількості одночасних з'єднань з базою даних до 10).

5.2 Що таке потокобезпечні (thread-safe) структури даних? Наведіть приклади.

Це структури даних, які гарантують коректну роботу (відсутність гонок даних або пошкодження пам'яті) при одночасному зверненні до них з кількох потоків. У Rust це `Arc<Mutex<T>>` або `RwLock<T>`. На рівні колекцій це потокобезпечні черги (наприклад, канали `mpsc` або черги з бібліотеки `crossbeam`).

5.3 Поясніть різницю між песимістичною та оптимістичною синхронізацією.

Песимістична синхронізація передбачає, що конфлікт точно станеться, тому потік блокує ресурс (захоплює м'ютекс) до початку роботи з ним. Оптимістична синхронізація передбачає, що конфлікти рідкісні: потік виконує обчислення локально, а перед записом результату перевіряє, чи не змінив дані хтось інший (зазвичай за допомогою атомарних операцій Compare-And-Swap). Якщо змінив — транзакція скасовується і повторюється.

5.4 Що таке lock-free та wait-free алгоритми?

Це алгоритми неблокуючої синхронізації. Lock-free гарантує, що система в цілому завжди прогресує (хоча б один потік виконує корисну роботу), навіть якщо інші потоки призупинені ОС. Wait-free є більш суворою вимогою: він гарантує, що кожен потік завершить свою операцію за скінченну, передбачувану кількість кроків, незалежно від дій інших потоків.

5.5 Поясніть патерни Producer-Consumer, Pipeline, Map-Reduce, Fork-Join, їх реалізацію.

- Producer-Consumer: Один або кілька потоків-виробників генерують дані і кладуть їх у загальну чергу, з якої потоки-споживачі їх забирають. Реалізовано через `mpsc::channel`.
- Pipeline: Ланцюг обробки, де дані передаються від однієї спеціалізованої стадії до іншої. Реалізовано через кілька з'єднаних між собою каналів.
- Map-Reduce: Розподіл задачі на незалежні частини (Map), їх паралельна обробка та злиття результатів (Reduce). У Rayon реалізовано через `.par_iter().map().reduce()`.

- Fork-Join: Рекурсивний поділ великої задачі на менші (Fork) до досягнення базового розміру, їх паралельне виконання та об'єднання результатів (Join). Реалізовано через функцію `rayon::join`.

5.6 Поясніть можливі відмінності в реалізації патернів відносно різних фреймворків.

Реалізація сильно залежить від філософії мови. У C++ (OpenMP) Map-Reduce або Worker Pool часто реалізуються декларативно, через прагми `#pragma omp parallel for`, де компілятор сам керує потоками. У Java використовується об'єктно-орієнтований підхід з класом `ForkJoinPool`. У Go патерни Pipeline та Producer-Consumer є "нативними" і реалізуються дуже легко через вбудовані горутини (`goroutines`) та канали (`chan`). Rust змушує розробника явно описувати володіння даними під час паралелізму, що робить код більш багатослівним (потреба у `std::sync::Arc`), але математично гарантує відсутність помилок сегментації (segfaults) під час виконання.

6 ВИСНОВКИ

У ході виконання лабораторної роботи було реалізовано та досліджено патерни паралельного програмування.

1. Доведено, що гіперпараметри критично впливають на продуктивність. Наприклад, у патерні Fork-Join занадто малий поріг рекурсії призводить до того, що накладні витрати на керування потоками перевищують час самих обчислень. Для Worker Pool оптимальна кількість потоків дорівнює кількості логічних ядер процесора.
2. Продемонстровано вплив закону Амдала на практиці: неможливість розпаралелити функцію сортування призвела до повної деградації паралельного пулу потоків у задачі пошуку медіани.
3. Аналіз показав, що Map-Reduce є найбільш лаконічним і швидким патерном для агрегації даних, Fork-Join ідеальний для ієрархічних математичних обчислень, а Producer-Consumer забезпечує відмінний розподіл навантаження для незалежних задач (як обробка окремих файлів). Pipeline виправдовує себе при гетерогенних навантаженнях, проте вимагає ретельного управління пам’яттю (обмежені канали) та уваги до уникнення взаємних блокувань.

Додаток А
Лістинг коду

A.1 src/bin/html.rs

```
use rayon::prelude::*;

fn process_file(path: &std::path::PathBuf, pattern: &regex::Regex) ->
std::collections::HashMap<String, usize> {
    let mut counts = std::collections::HashMap::new();
    let content = std::fs::read_to_string(path).unwrap();
    for cap in pattern.captures_iter(&content) {
        let tag = cap[1].to_lowercase();
        *counts.entry(tag).or_insert(0) += 1;
    }
    counts
}

fn merge_maps(mut map1: std::collections::HashMap<String, usize>, map2:
std::collections::HashMap<String, usize>) ->
std::collections::HashMap<String, usize> {
    for (key, val) in map2 {
        *map1.entry(key).or_insert(0) += val;
    }
    map1
}

fn run_sequential(paths: &[std::path::PathBuf], pattern: &regex::Regex)
-> std::collections::HashMap<String, usize> {
    let mut total_counts = std::collections::HashMap::new();
    for path in paths {
        let file_counts = process_file(path, pattern);
        total_counts = merge_maps(total_counts, file_counts);
    }
    total_counts
}

fn run_map_reduce(paths: &[std::path::PathBuf], pattern: &regex::Regex)
-> std::collections::HashMap<String, usize> {
    paths.par_iter()
        .map(|path| process_file(path, pattern))
        .reduce( || std::collections::HashMap::new(), |map1, map2|
merge_maps(map1, map2))
}

fn run_fork_join(paths: &[std::path::PathBuf], pattern: &regex::Regex)
-> std::collections::HashMap<String, usize> {
    if paths.len() <= 8 {
```

```
        return run_sequential(paths, pattern);
    }
    let mid = paths.len() / 2;
    let (left, right) = paths.split_at(mid);
    let (left_result, right_result) = rayon::join(
        || run_fork_join(left, pattern),
        || run_fork_join(right, pattern)
    );
    merge_maps(left_result, right_result)
}

fn run_worker_pool(paths: &[std::path::PathBuf], num_workers: usize,
pattern: &regex::Regex) -> std::collections::HashMap<String, usize> {
    let (job_tx, job_rx) =
std::sync::mpsc::channel::<std::path::PathBuf>();
    let (result_tx, result_rx) =
std::sync::mpsc::channel::<std::collections::HashMap<String, usize>>();

    let job_rx = std::sync::Arc::new(std::sync::Mutex::new(job_rx));

    std::thread::scope(|s| {
        for _ in 0..num_workers {
            let rx = std::sync::Arc::clone(&job_rx);
            let tx = result_tx.clone();

            s.spawn(move || {
                loop {
                    let path = {
                        let lock = rx.lock().unwrap();
                        match lock.recv() {
                            Ok(p) => p,
                            Err(_) => break,
                        }
                    };

                    let counts = process_file(&path, pattern);
                    tx.send(counts).unwrap();
                }
            });
        }

        std::mem::drop(result_tx);

        for path in paths {
            job_tx.send(path.clone()).unwrap();
        }

        std::mem::drop(job_tx);
```

```
        let mut total_counts = std::collections::HashMap::new();

        for result in result_rx {
            total_counts = merge_maps(total_counts, result);
        }

        total_counts
    })
}

use std::io::Write;

fn main() {
    let pattern = regex::Regex::new(r"<([a-zA-Z0-9]+)[^>]*>").unwrap();
    let paths = std::fs::read_dir("build/wikipedia").unwrap().map(|f|
f.unwrap().path()).collect::<Vec<_>>();
    let mut results = std::fs::File::create("build/html.csv").unwrap();

    let start = std::time::Instant::now();
    let res = run_sequential(&paths, &pattern);

    writeln!(results, "sequential,{},{}",
start.elapsed().as_secs_f64(), res.len()).unwrap();

    let start = std::time::Instant::now();
    let res = run_map_reduce(&paths, &pattern);
    writeln!(results, "map-reduce,{},{}",
start.elapsed().as_secs_f64(), res.len()).unwrap();

    let start = std::time::Instant::now();
    let res = run_fork_join(&paths, &pattern);
    writeln!(results, "fork-join,{},{}", start.elapsed().as_secs_f64(),
res.len()).unwrap();

    let start = std::time::Instant::now();
    let num_cpus = std::thread::available_parallelism().unwrap().get();
    let res = run_worker_pool(&paths, num_cpus, &pattern);
    writeln!(results, "worker-pool,{},{}",
start.elapsed().as_secs_f64(), res.len()).unwrap();
}
```

A.2 src/bin/matrix.rs

```
use rayon::prelude::*;

fn run_sequential(a: &[f64], b: &[f64], n: usize) -> std::vec::Vec<f64>
{
    let mut c = std::vec![0.0; n * n];
```

```
    for i in 0..n {
        for j in 0..n {
            let mut sum = 0.0;
            for k in 0..n {
                sum += a[i * n + k] * b[k * n + j];
            }
            c[i * n + j] = sum;
        }
    }
    c
}

fn run_map_reduce(a: &[f64], b: &[f64], n: usize) -> std::vec::Vec<f64>
{
    (0..n).into_par_iter().flat_map(|i| {
        let mut row = std::vec![0.0; n];
        for j in 0..n {
            let mut sum = 0.0;
            for k in 0..n {
                sum += a[i * n + k] * b[k * n + j];
            }
            row[j] = sum;
        }
        row
    }).collect()
}

fn fork_join_compute(a: &[f64], b: &[f64], n: usize, row_start: usize,
row_end: usize) -> std::vec::Vec<f64> {
    if row_end - row_start <= 16 {
        let mut chunk = std::vec![0.0; (row_end - row_start) * n];
        for i in row_start..row_end {
            for j in 0..n {
                let mut sum = 0.0;
                for k in 0..n {
                    sum += a[i * n + k] * b[k * n + j];
                }
                chunk[(i - row_start) * n + j] = sum;
            }
        }
        return chunk;
    }

    let mid = row_start + (row_end - row_start) / 2;
    let (mut left, right) = rayon::join(
        || fork_join_compute(a, b, n, row_start, mid),
        || fork_join_compute(a, b, n, mid, row_end)
    );
```

```
    left.extend(right);
    left
}

fn run_fork_join(a: &[f64], b: &[f64], n: usize) -> std::vec::Vec<f64>
{
    fork_join_compute(a, b, n, 0, n)
}

fn run_worker_pool(a: &[f64], b: &[f64], n: usize, num_workers: usize)
-> std::vec::Vec<f64> {
    let (job_tx, job_rx) = std::sync::mpsc::channel::<usize>();
    let (result_tx, result_rx) = std::sync::mpsc::channel::<(usize,
std::vec::Vec<f64>)>();

    let job_rx = std::sync::Arc::new(std::sync::Mutex::new(job_rx));

    std::thread::scope(|s| {
        for _ in 0..num_workers {
            let rx = std::sync::Arc::clone(&job_rx);
            let tx = result_tx.clone();

            s.spawn(move || {
                loop {
                    let row_idx = {
                        let lock = rx.lock().unwrap();
                        match lock.recv() {
                            Ok(idx) => idx,
                            Err(_) => break,
                        }
                    };

                    let mut row = std::vec![0.0; n];
                    for j in 0..n {
                        let mut sum = 0.0;
                        for k in 0..n {
                            sum += a[row_idx * n + k] * b[k * n + j];
                        }
                        row[j] = sum;
                    }
                    tx.send((row_idx, row)).unwrap();
                }
            });
        }

        std::mem::drop(result_tx);
```

```
        for i in 0..n {
            job_tx.send(i).unwrap();
        }

        std::mem::drop(job_tx);

        let mut c = std::vec![0.0; n * n];
        for (row_idx, row_data) in result_rx {
            let start = row_idx * n;
            c[start..start + n].copy_from_slice(&row_data);
        }

        c
    })
}

fn main() {
    let n = 3000;
    let size = n * n;
    let mut a = std::vec::Vec::with_capacity(size);
    let mut b = std::vec::Vec::with_capacity(size);
    let mut val: f64 = 0.5;

    for _ in 0..size {
        val = (val * 1.523).fract();
        a.push(val * 10.0);

        val = (val * 1.523).fract();
        b.push(val * 10.0);
    }

    std::fs::create_dir_all("build").unwrap();
    let mut results = std::fs::File::create("build/
matrix.csv").unwrap();

    let start = std::time::Instant::now();
    let res_seq = run_sequential(&a, &b, n);
    let sum_seq: f64 = res_seq.iter().sum();
    std::io::Write::write_fmt(&mut results, format_args!("sequential,
{},{:.2}\n", start.elapsed().as_secs_f64(), sum_seq)).unwrap();

    let start = std::time::Instant::now();
    let res_mr = run_map_reduce(&a, &b, n);
    let sum_mr: f64 = res_mr.iter().sum();
    std::io::Write::write_fmt(&mut results, format_args!("map-reduce,
{},{:.2}\n", start.elapsed().as_secs_f64(), sum_mr)).unwrap();

    let start = std::time::Instant::now();
```

```
    let res_fj = run_fork_join(&a, &b, n);
    let sum_fj: f64 = res_fj.iter().sum();
    std::io::Write::write_fmt(&mut results, format_args!("fork-join,{},
{:.2}\n", start.elapsed().as_secs_f64(), sum_fj)).unwrap();

    let start = std::time::Instant::now();
    let num_cpus = std::thread::available_parallelism().unwrap().get();
    let res_wp = run_worker_pool(&a, &b, n, num_cpus);
    let sum_wp: f64 = res_wp.iter().sum();
    std::io::Write::write_fmt(&mut results, format_args!("worker-pool,
{},{:.2}\n", start.elapsed().as_secs_f64(), sum_wp)).unwrap();
}
```

A.3 src/bin/numbers.rs

```
use rayon::prelude::*;
use std::io::Write;

fn process_chunk(chunk: &[f64]) -> (f64, f64, f64, usize) {
    let mut min = f64::INFINITY;
    let mut max = f64::NEG_INFINITY;
    let mut sum = 0.0;
    let count = chunk.len();

    for &val in chunk {
        if val < min { min = val; }
        if val > max { max = val; }
        sum += val;
    }

    (min, max, sum, count)
}

fn merge_stats(a: (f64, f64, f64, usize), b: (f64, f64, f64, usize)) ->
(f64, f64, f64, usize) {
    (
        a.0.min(b.0),
        a.1.max(b.1),
        a.2 + b.2,
        a.3 + b.3,
    )
}

fn get_median_sequential(data: &[f64]) -> f64 {
    let mut sorted = data.to_vec();
    sorted.sort_unstable_by(|a, b| a.partial_cmp(b).unwrap());
    let mid = sorted.len() / 2;
    if sorted.len() % 2 == 0 {
        (sorted[mid - 1] + sorted[mid]) / 2.0
```

```
    } else {
        sorted[mid]
    }
}

fn get_median_parallel(data: &[f64]) -> f64 {
    let mut sorted = data.to_vec();
    sorted.par_sort_unstable_by(|a, b| a.partial_cmp(b).unwrap());
    let mid = sorted.len() / 2;
    if sorted.len() % 2 == 0 {
        (sorted[mid - 1] + sorted[mid]) / 2.0
    } else {
        sorted[mid]
    }
}

fn run_sequential(data: &[f64]) -> (f64, f64, f64, f64) {
    let stats = process_chunk(data);
    let mean = stats.2 / stats.3 as f64;
    let median = get_median_sequential(data);
    (stats.0, stats.1, median, mean)
}

fn run_map_reduce(data: &[f64]) -> (f64, f64, f64, f64) {
    let (min, max, sum, count) = data.par_chunks(10000)
        .map(|chunk| process_chunk(chunk))
        .reduce(|| (std::f64::INFINITY, std::f64::NEG_INFINITY, 0.0,
0), merge_stats);

    let mean = sum / count as f64;
    let median = get_median_parallel(data);
    (min, max, median, mean)
}

fn fork_join_stats(data: &[f64]) -> (f64, f64, f64, usize) {
    if data.len() <= 10000 {
        return process_chunk(data);
    }
    let mid = data.len() / 2;
    let (left, right) = data.split_at(mid);
    let (left_result, right_result) = rayon::join(
        || fork_join_stats(left),
        || fork_join_stats(right)
    );
    merge_stats(left_result, right_result)
}

fn run_fork_join(data: &[f64]) -> (f64, f64, f64, f64) {
```

```
    let stats = fork_join_stats(data);
    let mean = stats.2 / stats.3 as f64;
    let median = get_median_parallel(data);
    (stats.0, stats.1, median, mean)
}

fn run_worker_pool(data: &[f64], num_workers: usize) -> (f64, f64, f64,
f64) {
    let (job_tx, job_rx) = std::sync::mpsc::channel::<&[f64]>();
    let (result_tx, result_rx) = std::sync::mpsc::channel::<(f64, f64,
f64, usize)>();

    let job_rx = std::sync::Arc::new(std::sync::Mutex::new(job_rx));

    let final_stats = std::thread::scope(|s| {
        for _ in 0..num_workers {
            let rx = std::sync::Arc::clone(&job_rx);
            let tx = result_tx.clone();

            s.spawn(move || {
                loop {
                    let chunk = {
                        let lock = rx.lock().unwrap();
                        match lock.recv() {
                            Ok(c) => c,
                            Err(_) => break,
                        }
                    };

                    let stats = process_chunk(chunk);
                    tx.send(stats).unwrap();
                }
            });
        }

        std::mem::drop(result_tx);

        let chunk_size = (data.len() + num_workers - 1) / num_workers;
        for chunk in data.chunks(chunk_size) {
            job_tx.send(chunk).unwrap();
        }

        std::mem::drop(job_tx);

        let mut total_stats = (std::f64::INFINITY,
std::f64::NEG_INFINITY, 0.0, 0);

        for result in result_rx {
```

```
            total_stats = merge_stats(total_stats, result);
        }

        total_stats
    });

    let mean = final_stats.2 / final_stats.3 as f64;
    let median = get_median_sequential(data);

    (final_stats.0, final_stats.1, median, mean)
}

fn main() {
    let size = 100_000_000;
    let mut data = Vec::with_capacity(size);
    let mut val: f64 = 0.5;

    for _ in 0..size {
        val = (val * 1.523).fract();
        data.push(val * 1000.0);
    }

    std::fs::create_dir_all("build").unwrap();
    let mut results = std::fs::File::create("build/
numbers.csv").unwrap();

    let start = std::time::Instant::now();
    let res_seq = run_sequential(&data);
    writeln!(results, "sequential,{},{},{},{},{}",
start.elapsed().as_secs_f64(), res_seq.0, res_seq.1, res_seq.2,
res_seq.3).unwrap();

    let start = std::time::Instant::now();
    let res_mr = run_map_reduce(&data);
    writeln!(results, "map-reduce,{},{},{},{},{}",
start.elapsed().as_secs_f64(), res_mr.0, res_mr.1, res_mr.2,
res_mr.3).unwrap();

    let start = std::time::Instant::now();
    let res_fj = run_fork_join(&data);
    writeln!(results, "fork-join,{},{},{},{},{}",
start.elapsed().as_secs_f64(), res_fj.0, res_fj.1, res_fj.2,
res_fj.3).unwrap();

    let start = std::time::Instant::now();
    let num_cpus = std::thread::available_parallelism().unwrap().get();
    let res_wp = run_worker_pool(&data, num_cpus);
    writeln!(results, "worker-pool,{},{},{},{},{}",
```

```
start.elapsed().as_secs_f64(), res_wp.0, res_wp.1, res_wp.2,
res_wp.3).unwrap();
}
```

A.4 src/bin/video.rs

```
use image::{GenericImage, Rgba, GenericImageView};
use std::io::Write;

fn apply_watermark(mut img: image::DynamicImage) -> image::DynamicImage
{
    let (width, height) = img.dimensions();
    let watermark_size = 20;

    for x in 0..watermark_size {
        for y in 0..watermark_size {
            if x < width && y < height {
                img.put_pixel(x, y, Rgba([255, 0, 0, 255]));
            }
        }
    }
    img
}

fn generate_dummy_frames(input_dir: &str, count: usize) ->
Vec<std::path::PathBuf> {
    std::fs::create_dir_all(input_dir).unwrap();
    let mut paths = Vec::new();

    for i in 0..count {
        let path = std::path::PathBuf::from(format!("{}/
frame_{:03}.png", input_dir, i));
        let img = image::ImageBuffer::from_pixel(100, 100, Rgba([(i %
255) as u8, 100, 100, 255]));
        img.save(&path).unwrap();
        paths.push(path);
    }

    paths
}

fn run_producer_consumer(paths: &[std::path::PathBuf], output_dir:
&str, num_workers: usize) {
    let (job_tx, job_rx) =
std::sync::mpsc::channel::<std::path::PathBuf>();
    let job_rx = std::sync::Arc::new(std::sync::Mutex::new(job_rx));
    let out_dir = output_dir.to_string();

    std::thread::scope(|s| {
```

```
        for _ in 0..num_workers {
            let rx = std::sync::Arc::clone(&job_rx);
            let out = out_dir.clone();

            s.spawn(move || {
                loop {
                    let path = {
                        let lock = rx.lock().unwrap();
                        match lock.recv() {
                            Ok(p) => p,
                            Err(_) => break, // Breaks when job_tx is
dropped
                        }
                    };

                    if let Ok(img) = image::open(&path) {
                        let filtered = img.grayscale();
                        let watermarked = apply_watermark(filtered);

                        let name =
path.file_name().unwrap().to_string_lossy().into_owned();
                        let out_path =
std::path::Path::new(&out).join(name);
                        watermarked.save(out_path).unwrap();
                    }
                }
            });
        }

        for path in paths {
            job_tx.send(path.clone()).unwrap();
        }

        // FIX: Tell the workers we are done sending jobs
        std::mem::drop(job_tx);
    });
}

fn run_pipeline(paths: &[std::path::PathBuf], output_dir: &str) {
    let (tx1, rx1) =
std::sync::mpsc::sync_channel::<std::path::PathBuf>(10);
    let (tx2, rx2) = std::sync::mpsc::sync_channel::<(String,
image::DynamicImage)>(10);
    let (tx3, rx3) = std::sync::mpsc::sync_channel::<(String,
image::DynamicImage)>(10);

    let out_dir = output_dir.to_string();
```

```
    std::thread::scope(|s| {
        s.spawn(move || {
            for path in rx1 {
                if let Ok(img) = image::open(&path) {
                    let name =
path.file_name().unwrap().to_string_lossy().into_owned();
                    tx2.send((name, img)).unwrap();
                }
            }
        });

        s.spawn(move || {
            for (name, img) in rx2 {
                let filtered = img.grayscale();
                tx3.send((name, filtered)).unwrap();
            }
        });

        s.spawn(move || {
            for (name, img) in rx3 {
                let watermarked = apply_watermark(img);
                let out_path =
std::path::Path::new(&out_dir).join(name);
                watermarked.save(out_path).unwrap();
            }
        });

        for path in paths {
            tx1.send(path.clone()).unwrap();
        }

        std::mem::drop(tx1);
    });
}

fn main() {
    let input_dir = "build/frames_in";
    let output_dir_pc = "build/frames_out_pc";
    let output_dir_pipe = "build/frames_out_pipe";

    std::fs::create_dir_all(output_dir_pc).unwrap();
    std::fs::create_dir_all(output_dir_pipe).unwrap();

    println!("generating dummy frames...");
    let paths = generate_dummy_frames(input_dir, 100000);
    let mut results = std::fs::File::create("build/
image_processing.csv").unwrap();
```

```
    let num_cpus = std::thread::available_parallelism().unwrap().get();

    println!("running producer-consumer worker pool...");
    let start = std::time::Instant::now();
    run_producer_consumer(&paths, output_dir_pc, num_cpus);
    writeln!(results, "producer-consumer,{}\n",
start.elapsed().as_secs_f64()).unwrap();

    println!("running 3-stage pipeline...");
    let start = std::time::Instant::now();
    run_pipeline(&paths, output_dir_pipe);
    writeln!(results, "pipeline,{}\n",
start.elapsed().as_secs_f64()).unwrap();

    println!("done...");
}
```

A.5 src/bin/wikipedia.rs

```
use reqwest;
use reqwest::IntoUrl;
use std::io::Write;

fn needs_rebuild(out_path: &dyn AsRef<std::path::Path>, in_paths:
&[&dyn AsRef<std::path::Path>]) -> bool {
    if !out_path.as_ref().exists() {
        true
    } else {
        let out_modified_time =
out_path.as_ref().metadata().unwrap().modified().unwrap();
        in_paths.iter().map(|p| p.as_ref().metadata().unwrap()).any(|m|
m.modified().unwrap() > out_modified_time)
    }
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let user_agent = "MyRustWikiBot/1.0 (contact: your-
email@example.com)";
    let client =
reqwest::Client::builder().user_agent(user_agent).build()?;

    let url = "https://en.wikipedia.org/wiki/Special:Random";
    let build_dir = std::path::Path::new("build");
    if needs_rebuild(&build_dir, &[]) {
        std::fs::create_dir(&build_dir).unwrap();
    }
    let mut pages_path = build_dir.join("wikipedia");
    let _ = std::fs::remove_dir_all(&pages_path);
```

```
    std::fs::create_dir(&pages_path).unwrap();

    for i in (0..1000) {
        let page =
client.get(url).send().await.unwrap().text().await.unwrap();
        let page_path =
pages_path.join(i.to_string()).with_extension("html");
        let mut file = std::fs::File::create(&page_path).unwrap();
        file.write_all(page.as_bytes()).unwrap();
        println!("{i} downloaded");

tokio::time::sleep(std::time::Duration::from_millis(500)).await;
    }
    Ok(())
}
```